

Haan bhai, bilkul! "Reverse a string" (string ko ulta karna) coding aur Data Structures & Algorithms (DSA) ki journey ka sabse pehla aur sabse important kadam hai. Isko acche se samajhna logic building ke liye bahut zaroori hai.

Chalo isko detail mein, step-by-step samajhte hain.

Basic Concept Kya Hai?

Bahut simple hai: Jo string input mein aayi hai, uske characters ko peeche se aage ki taraf likhna hai.

- **Input:** "HELLO"
- **Output:** "OLLEH"

Sabse Best Tarika: The "Two-Pointer" Approach

Interviews mein aur logic build karne ke liye hamesha **Two-Pointer Approach** sabse best maani jati hai. Isme hum bina koi nayi string banaye, usi string ke andar letters ko swap (aas-pas badalna) karte hain.

Logic aise kaam karta hai:

1. Ek pointer (index) rakho string ke bilkul shuru mein. Isko naam dete hain **left** (index 0).
2. Doosra pointer rakho string ke bilkul aakhiri character par. Isko naam dete hain **right** (index **length - 1**).
3. Ab **left** aur **right** wale characters ki jagah aapas mein badal do (Swap).
4. Uske baad, **left** ko ek kadam aage badhao (**left++**) aur **right** ko ek kadam peeche laao (**right--**).
5. Ye process tab tak repeat karo jab tak **left** index, **right** index se chhota hai (**left < right**). Jaise hi dono beech mein aakar mil jayein, matlab string reverse ho chuki hai!

Dry Run (Example ke saath)

Maan lo hamari string hai "**BHAI**". Isme 4 characters hain. Indices honge: 0, 1, 2, 3.

Index	0	1	2	3
Character	B	H	A	I

- **Step 1:** **left** = 0 (B), **right** = 3 (I).
- Swap karo: **I** aur **B** ko.
- String ban gayi: "**IHAB**"
- Ab **left** ban gaya 1, **right** ban gaya 2.
- **Step 2:** **left** = 1 (H), **right** = 2 (A).
- Swap karo: **A** aur **H** ko.
- String ban gayi: "**IAHB**"

- Ab **left** ban gaya 2, **right** ban gaya 1.
 - **Step 3:** Ab **left** (2), **right** (1) se bada ho gaya hai. Yahan hum ruk jayenge.
 - **Final Output:** "IAHB"
-

Code Kaise Likhte Hain?

Yahan 2 popular languages mein iska basic code hai:

1. C++ Code (Two-Pointer)

```
#include <iostream>
#include <string>
using namespace std;

void reverseString(string &s) {
    int left = 0;
    int right = s.length() - 1;

    while (left < right) {
        // Characters ko swap kar rahe hain
        swap(s[left], s[right]);
        // Pointers ko aage-peeche move karo
        left++;
        right--;
    }
}

int main() {
    string str = "BHAI";
    reverseString(str);
    cout << "Reversed String: " << str << endl;
    return 0;
}
```

Time aur Space Complexity

- **Time Complexity:** $O(N)$ - Kyunki hum string ke aadhe characters tak loop chalte hain ($N/2$ steps lagte hain, jo $O(N)$ hi hota hai). N yahan string ki length hai.
- **Space Complexity:** $O(1)$ - Kyunki humne koi nayi extra string nahi banayi, bas exist karne wali string mein hi badlaav kiya hai (In-place sorting).

Agar is logic mein koi bhi line ya step samajh na aaya ho, toh aaram se bata de bhai, main aur asaan bhasha mein tod ke samjha dunga!

Haan bhai, bilkul! Lagta hai aap "**Defanging an IP Address**" problem ki baat kar rahe ho (jo LeetCode ka ek bahut popular aur beginner-friendly question hai). String manipulation seekhne ke liye yeh ek perfect problem

hai.

Chalo isko detail mein aur aasan bhasha mein samajhte hain.

Basic Concept: Defanging ka matlab kya hai?

Network security mein, malicious (khatarnak) IP addresses ko clickable link banne se rokne ke liye unhe "defang" kiya jata hai. Programming terms mein iska simple sa matlab yeh hai:

Aapko ek IP address string format mein diya jayega. Aapko us string mein **har . (dot) ko [.] se replace karna hai.**

- **Input:** "1.1.1.1"
- **Output:** "1[.]1[.]1[.]1"

Logic Kya Hoga?

Hum seedha original string ko modify nahi karenge kyunki strings mein beech mein naye characters daalna (insert karna) thoda slow aur complex hota hai. Uske bajaye hum ek **nayi string banayenge.**

Step-by-Step Approach:

1. Ek nayi empty string banao jisme hum apna final answer rakhenge. Isko **ans** maan lete hain.
2. Di gayi input string (**address**) par ek loop lagao aur har character ko ek-ek karke check karo.
3. **Condition 1:** Agar character ek dot (.) hai, toh hamari **ans** string mein "[.]" add kar do.
4. **Condition 2:** Agar character koi number hai, toh use bina kisi change ke waisa ka waisa hi **ans** string mein add kar do.
5. Pura loop khatam hone ke baad **ans** ko return kar do.

Dry Run (Example ke saath)

Maan lo hamara input hai: "1.2"

Ek khali string banate hain: **ans** = ""

Step	Character	Kya yeh . hai?	ans mein kya add hoga?	ans ki nayi value
1	'1'	Nahi	'1'	"1"
2	'.'	Haan	"[.]"	"1[.]"
3	'2'	Nahi	'2'	"1[.]2"

Final Output: "1[.]2"

C++ Code

Yahan iska pura C++ code hai jise aap seedha run kar sakte ho:

```
#include <iostream>
#include <string>
using namespace std;

string defangIPAddr(string address) {
    // Ek nayi string banayi answer store karne ke liye
    string ans = "";

    // Input string ke har character par loop chalayenge
    for (int i = 0; i < address.length(); i++) {

        // Agar current character dot hai
        if (address[i] == '.') {
            ans += "[.]"; // Nayi string mein [.] add kar do
        }
        // Agar current character number hai
        else {
            ans += address[i]; // Waisa ka waisa add kar do
        }
    }

    // Final defanged string return kar do
    return ans;
}

int main() {
    string ip = "192.168.1.1";
    string result = defangIPAddr(ip);

    cout << "Original IP: " << ip << endl;
    cout << "Defanged IP: " << result << endl;

    return 0;
}
```

Time aur Space Complexity

- **Time Complexity:** $O(N)$ — Yahan N string ki total length hai. Hum string ke har character par sirf ek baar ja rahe hain (ek single loop hai), isliye time complexity $O(N)$ hogi.
- **Space Complexity:** $O(N)$ — Humne apna answer store karne ke liye ek nayi string (**ans**) banayi hai, jo lagbhag original string jitni hi jagah legi. Isliye memory (space) bhi $O(N)$ use hogi.

Bhai, Pangram checking bhi ek dum mast aur interview-favourite question hai. Yeh bohot simple hai agar aap iska logical core samajh jao.

Chalo isko bhi poori detail mein step-by-step samajhte hain!

Pangram Kya Hota Hai?

Pangram ek aisi string (ya sentence) hoti hai jisme **English alphabet ke saare 26 letters (a se z tak) kam se kam ek baar zaroor aayein**. Chahe lowercase ho ya uppercase, bas saare 26 letters hone chahiye.

- **Example 1:** "The quick brown fox jumps over the lazy dog"
 - Yeh ek **Pangram** hai, kyunki isme **a** se leke **z** tak saare letters aa rahe hain.
 - **Example 2:** "leetcode"
 - Yeh **Pangram nahi hai**, kyunki isme sirf **l, e, t, c, o, d** hi hain, baaki akshar missing hain.
-

Sabse Efficient Approach: Boolean Array (Size 26)

Hamein bas yeh track rakhna hai ki kaun-kaun sa letter string mein aa chuka hai. Iske liye hum 26 size ka ek khali **Boolean Array** ya **Visited Array** banate hain (kyunki alphabets 26 hote hain).

Logic aise kaam karega:

1. Ek boolean array banao `visited[26]` naam ka aur shuru mein sabko `false` set kar do. Yahan index `0` ka matlab `a`, index `1` ka matlab `b`, aur index `25` ka matlab `z` hoga.
 2. String ke har character ko check karo:
 - Agar character uppercase hai (`A` to `Z`), toh use lowercase mein badal lo.
 - Agar character koi letter hai, toh uska index nikalne ke liye `character - 'a'` karo (C++ mein ASCII value se index mil jata hai).
 - Us index par `visited[index] = true` kar do.
 3. Poori string ghoomne ke baad, apne `visited` array ko check karo. Agar koi bhi index `false` reh gaya, iska matlab woh letter string mein nahi tha → **Not a Pangram**.
 4. Agar saare 26 spots `true` hain → **It's a Pangram!**
-

Dry Run (Chota Example)

Maan lo string hai: "Abc" Hamara `visited` array 26 size ka hai (sab `false` hain).

- **Step 1:** Pehla character '`A`'. Lowercase kiya → '`a`'.
- `Index = 'a' - 'a' = 0`.
- `visited[0] = true`.
- **Step 2:** Dusra character '`b`'.
- `Index = 'b' - 'a' = 1`.
- `visited[1] = true`.
- **Step 3:** Tisra character '`c`'.
- `Index = 'c' - 'a' = 2`.
- `visited[2] = true`.

Loop khatam. Ab hum **visited** array check karenge. Index 3 se lekar 25 tak abhi bhi **false** hain (kyunki **d** se **z** tak kuch nahi mila). Isliye yeh **Pangram nahi hai**.

C++ Code

Yahan iska sabse clean aur optimized C++ code hai:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

bool checkIfPangram(string sentence) {
    // 26 size ka boolean array banaya, sabko false rakha
    vector<bool> visited(26, false);

    // String ke har character ko check karenge
    for (char ch : sentence) {
        // Agar character uppercase hai, toh lowercase banao
        if (ch >= 'A' && ch <= 'Z') {
            ch = ch + 32; // Ya fir tolower(ch) use kar sakte hain
        }

        // Sirf alphabets ko track karna hai (spaces ya numbers ko chhod dena hai)
        if (ch >= 'a' && ch <= 'z') {
            int index = ch - 'a'; // Character ka index nikalna (0 se 25)
            visited[index] = true; // Mark kar diya ki yeh letter mil gaya
        }
    }

    // Ab check karo ki koi letter chhoot toh nahi gaya
    for (int i = 0; i < 26; i++) {
        if (visited[i] == false) {
            return false; // Ek bhi letter missing mila toh false return karo
        }
    }

    return true; // Agar saare true hain toh Pangram hai
}

int main() {
    string str = "The quick brown fox jumps over the lazy dog";

    if (checkIfPangram(str)) {
        cout << "Haan bhai, yeh Pangram hai!" << endl;
    } else {
        cout << "Nahi bhai, yeh Pangram nahi hai." << endl;
    }

    return 0;
}
```

```
}
```

🕒 Time aur Space Complexity

- **Time Complexity:** $O(N)$ — Kyunki hum string ke har character par sirf ek baar ja rahe hain (N string ki length hai). Baaki jo array check karne wala loop hai woh hamesha fix 26 baar hi chalega, jo ki constant $O(1)$ hai.
- **Space Complexity:** $O(1)$ — Humne jo array banaya hai uski size hamesha 26 hi rahegi, chahe string kitni bhi badi ho jaye. Constant space use ho rahi hai.

(Pro Tip: Is question ko aap **Hash Set** se bhi kar sakte ho. String ke saare valid characters ko unique set mein daalte jao, aur aakhiri mein agar set ki size 26 ho gayi, toh woh Pangram hai!)

Haan bhai, bilkul! String sorting ka **Brute Force** tarika samajhte hain.

Brute force ka seedha matlab hota hai—sabse basic, bina kisi smart trick ya built-in function (`sort()`) ke, bilkul aamne-saamne compare karke logic banana. Iske liye hum **Nested Loops (do loops)** ka use karte hain, jise hum *Selection Sort* ya *Bubble Sort* ka logic bhi kehte hain.

Chalo isko poori detail mein step-by-step todte hain.

🎯 Basic Concept Kya Hai?

String ko aap characters ka ek array samajh sakte ho. Humein ise alphabetical order (A se Z ya a se z) mein arrange karna hai.

- **Input:** "DCBA"
- **Output:** "ABCD"

Brute Force Logic:

1. Hum pehle character par khade honge aur uske aage wale saare characters ko check karenge.
2. Agar aage koi aisa character milta hai jo hamare current character se chhota hai (ASCII value mein), toh hum un dono ko aapas mein **swap (badal)** denge.
3. Yeh process hum har ek character ke liye repeat karenge jab tak poori string sort nahi ho jaati.

🔍 Dry Run (Sabse Aasan Tarika Samajhne Ka)

Maan lo hamari string hai: "CADB" (Length $N = 4$)

Hum do loops chalayenge: `i` (outer loop) aur `j` (inner loop, jo `i+1` se shuru hoga).

- **Pass 1: `i = 0` (Character: 'C')**
- `j = 1` ('A'): Kya 'C' > 'A' hai? **Haan**. Swap karo! String bani → "ACDB"
- `j = 2` ('D'): Kya 'A' > 'D' hai? Nahi. No change.

- $j = 3$ ('B'): Kya 'A' > 'B' hai? Nahi. No change.
- *Pehle pass ke baad, sabse chhota character 'A' apni sahi jagah (0 index) par aa gaya.*
- **Pass 2: $i = 1$ (Character: 'C')**
- $j = 2$ ('D'): Kya 'C' > 'D' hai? Nahi. No change.
- $j = 3$ ('B'): Kya 'C' > 'B' hai? **Haan**. Swap karo! String bani $\rightarrow$ "ABCD"
- *Dusre pass ke baad, 'B' bhi apni sahi jagah par aa gaya.*
- **Pass 3: $i = 2$ (Character: 'D')**
- $j = 3$ ('C'): Kya 'D' > 'C' hai? **Haan**. Swap karo! String bani $\rightarrow$ "ABDC" (Wait, let's trace correctly: after pass 2 it was "ABCD", so $s[2]$ is 'C', $s[3]$ is 'D'. Is 'C' > 'D'? No. So it remains "ABCD").
- *Pura loop chalne ke baad hamari sorted string milegi: "ABCD".*

C++ Code (Brute Force)

Yahan nested loops ka use karke string sort karne ka code hai:

```
#include <iostream>
#include <string>
using namespace std;

void bruteForceSortString(string &s) {
    int n = s.length();

    // Outer loop: Ek-ek karke har position ko fix karega
    for (int i = 0; i < n - 1; i++) {

        // Inner loop: Baaki bache characters se compare karega
        for (int j = i + 1; j < n; j++) {

            // Agar pehle wala character baad wale se bada hai
            if (s[i] > s[j]) {
                // Toh dono ko swap kar do
                char temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }
}

int main() {
    string str = "CADB";

    cout << "Original String: " << str << endl;
    bruteForceSortString(str);
}
```



```
    cout << "Sorted String (Brute Force): " << str << endl;\n\n    return 0;\n}
```

Time aur Space Complexity

Kyunki yeh brute force hai, toh yeh thoda slow hota hai:

- **Time Complexity:** $O(N^2)$ — Yahan do nested loops chal rahe hain. Agar string ki length N hai, toh lagbhag $N \times N$ baar comparisons hote hain. Isliye badi strings ke liye yeh approach achhi nahi maani jaati.
- **Space Complexity:** $O(1)$ — Humne koi extra memory ya nayi string nahi banayi. Jo bhi jhol-jhal kiya, original string ke andar hi kiya (In-place algorithm).

*(Interview Tip: Agar interview mein poochhein toh pehle yeh brute force batao, fir bolna ki "Sir, isko hum $O(N \log N)$ mein optimize kar sakte hain `std::sort()` lagakar, ya fir agar sirf lowercase characters hain toh **Counting Sort** se $O(N)$ mein bhi kar sakte hain!")*

Bhai, brute force ($O(N^2)$) toh samajh gaye, par interviews mein asli maza tab aata hai jab aap interviewer ko **Optimized Approaches** batate ho.

String ko optimize tareeke se sort karne ke **2 bade dhasu tarike** hain. Chalo dono ko detail mein samajhte hain!

Approach 1: Built-in Sort (The Quick Way)

C++ ki standard library mein pehle se ek bahut powerful function hota hai: `sort()`. Yeh background mein **IntroSort** use karta hai (jo QuickSort, HeapSort aur Insertion Sort ka combo hai).

Logic:

Aapko bas string ka shuruati point (`s.begin()`) aur aakhiri point (`s.end()`) is function mein dena hota hai, aur yeh automatic efficient tareeke se characters ko unki ASCII value ke hisaab se arrange kar deta hai.

C++ Code:

```
#include <iostream>\n#include <string>\n#include <algorithm> // sort() ke liye zaroori hai\nusing namespace std;\n\nint main() {\n    string str = "CADB";\n\n    // Built-in sort function\n    sort(str.begin(), str.end());\n}
```

```
cout << "Sorted String (std::sort): " << str << endl; // Output: ABCD
return 0;
}
```

⌚ Complexity:

- **Time Complexity:** $O(N \log N)$ — Yeh brute force ke $O(N^2)$ se lakho guna zyada tez hai, khaaskar badi strings ke liye.
- **Space Complexity:** $O(\log N)$ — Internal sorting algorithm thodi si stack memory use karta hai recursion ke liye.

👑 Approach 2: Counting Sort / Frequency Array (The Ultimate $O(N)$ Way)

Interviewers ka sabse favourite optimization yeh hai. Agar interview mein diya ho ki string mein **sirf lowercase English letters (a se z)** hain, toh aap isko bina kisi comparison ke **Linear Time** mein sort kar sakte ho!

💡 Logic (No Comparison!):

Humein pata hai ki English mein sirf 26 alphabets hote hain. Toh hum har character ko aapas mein compare karne ke bajaye unki ginti (count) rakhlenge.

1. **Step 1:** 26 size ka ek integer array banao `count[26]` aur sabme 0 bhar do.
 2. **Step 2:** String par ek loop chalao. Jo bhi character mile, uske respect mein `count` array ki value badha do (Frequency nikal lo).
- Jaise agar 'a' mila toh `count[0]++`, 'b' mila toh `count[1]++`.
3. **Step 3:** Ab ek naya loop chalao 0 se 25 tak (yaani a se z tak). Agar kisi index par count 0 se bada hai, toh us character ko utni baar apni final string mein likhte jao.

🔍 Dry Run (Example: "baba")

1. Frequency Counter:

- 'b' aaya → `count[1] = 1`
- 'a' aaya → `count[0] = 1`
- 'b' aaya → `count[1] = 2`
- 'a' aaya → `count[0] = 2`
- Baaki sab index 0 hain.

2. Rebuilding String:

- Index 0 ('a'): Count 2 hai → String mein "aa" joda.
- Index 1 ('b'): Count 2 hai → String mein "bb" joda.
- **Final String:** "aabb" (Sort ho gayi!)

💻 C++ Code:

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

string optimizedSortString(string s) {
    // 26 size ka frequency array (sabki starting value 0)
    vector<int> count(26, 0);

    // 1. Har character ki frequency count karo
    for (char ch : s) {
        count[ch - 'a']++; // 'ch - 'a' se 0 se 25 ke beech index milta hai
    }

    string sorted_ans = "";

    // 2. Frequency array se dobara sorted string taiyar karo
    for (int i = 0; i < 26; i++) {
        while (count[i] > 0) {
            sorted_ans += (i + 'a'); // Index ko wapas character mein badla
            count[i]--; // Count kam karte jao
        }
    }

    return sorted_ans;
}

int main() {
    string str = "edcba";
    cout << "Original: " << str << endl;
    cout << "Sorted ($O(N)$): " << optimizedSortString(str) << endl;
    return 0;
}

```

🕒 Complexity:

- **Time Complexity:** $O(N)$ — Humne string ko sirf ek baar scan kiya frequency nikalne ke liye, aur fir 26 size ka loop chalaya jo constant hai. Yeh sabse fast hai!
- **Space Complexity:** $O(1)$ — Jo `count` array humne banaya uski size hamesha 26 hi rahegi, chahe string ki length kitni bhi badi (10 lakh characters) kyun na ho. Constant space!

📊 Ek Najar Mein Comparison

Tarika (Approach)	Time Complexity	Space Complexity	Kab Use Karein?
Brute Force	$O(N^2)$	$O(1)$	Jab bilkul basic logic dikhana ho.

Tarika (Approach)	Time Complexity	Space Complexity	Kab Use Karein?
**Built-in <code>sort()</code> **	$O(N \log N)$	$O(\log N)$	General purpose, jab koi character constraint na ho (numbers, symbols sab mix hon).
Counting Sort	$O(N)$	$O(1)$	Best for Interviews! Jab characters limited hon (jaise sirf lowercase <code>a-z</code>).

Bhai, **Longest Palindromic Substring** ka brute force tarika dimaag kholne ke liye bohot badhiya hai. Iska seedha sa asool hai: "*Mehnat zyada, dimaag kam.*" Chalo isko poori detail mein step-by-step samajhte hain.

Problem Kya Hai?

Aapko ek string di jayegi, aur aapko uske andar se sabse lambi aisi **substring** (lagatar tukda) dhoondni hai jo ulta aur seedhi dono taraf se ek jaisi padhi jaye (Palindrome).

- **Input:** "`babad`"
- **Output:** "`bab`" (ya "`aba`" bhi ho sakta hai, kyunki dono ki length 3 hai aur dono palindrome hain).

Brute Force Logic (3 Loops Concept)

Brute force ka basic funda yeh hai ki hum string ke saare possible tukde (substrings) nikalenge, har ek tukde ko check karenge ki woh palindrome hai ya nahi, aur jo sabse bada hoga use yaad rakh lenge.

Isko karne ke liye hume **3 kaam** karne hote hain:

1. **Outer Loop (`i`):** Yeh substring ka starting point (shuruaat) tay karta hai.
2. **Inner Loop (`j`):** Yeh substring ka ending point (aakhiri) tay karta hai.
3. **Palindrome Check:** `i` se lekar `j` tak ki jo string bani, use ek alag function mein bhejkar check karenge ki woh palindrome hai ya nahi. Is check karne wale kaam mein ek aur loop lagta hai.

Isi wajah se isme total **3 loops** lag jaate hain.

Dry Run (Chota Example)

Maan lo hamari string hai: "`aba`" (Length $N = 3$)

Hum `i` (start) aur `j` (end) ke saare combinations banayenge:

- **Pass 1: `i = 0` (Character 'a' se shuru)**
- `j = 0` → Substring: "`a`" → Palindrome hai? **Haan**. (Length = 1)
- `j = 1` → Substring: "`ab`" → Palindrome hai? **Nahi**.
- `j = 2` → Substring: "`aba`" → Palindrome hai? **Haan**. (Length = 3) → *Ab tak ki sabse lambi string mili!*
- **Pass 2: `i = 1` (Character 'b' se shuru)**

- $j = 1 \rightarrow$ Substring: "b" $\rightarrow$ Palindrome hai? **Haan**. (Length = 1, par 3 se chhota hai toh ignore).
- $j = 2 \rightarrow$ Substring: "ba" $\rightarrow$ Palindrome hai? **Nahi**.
- **Pass 3: $i = 2$ (Character 'a' se shuru)**
- $j = 2 \rightarrow$ Substring: "a" $\rightarrow$ Palindrome hai? **Haan**. (Length = 1).

Final Answer: "aba" (Length 3)

C++ Code (Brute Force)

Yahan iska simple aur clear code hai:

```
#include <iostream>
#include <string>
using namespace std;

// Palindrome check karne ka helper function
bool isPalindrome(string &s, int start, int end) {
    while (start < end) {
        if (s[start] != s[end]) {
            return false; // Agar koi bhi character mismatch hua toh palindrome
nahi hai
        }
        start++;
        end--;
    }
    return true; // Agar loop poora chal gaya toh palindrome hai
}

string longestPalindromeBrute(string s) {
    int n = s.length();
    if (n == 0) return "";

    int max_len = 1; // Sabse lambi palindrome ki length track karne ke liye
    int start_index = 0; // Us lambi palindrome ka starting index yaad rakhne ke
    liye

    // 1. Outer loop: Substring ka starting point
    for (int i = 0; i < n; i++) {

        // 2. Inner loop: Substring ka ending point
        for (int j = i; j < n; j++) {

            // 3. Check karo kya i se j tak ki substring palindrome hai?
            if (isPalindrome(s, i, j)) {
                int current_len = j - i + 1; // Current substring ki length

                // Agar yeh ab tak ki sabse lambi palindrome hai, toh ise save
karlo
                if (current_len > max_len) {
```

```

        max_len = current_len;
        start_index = i;
    }
}

// Original string se woh sabse bada tukda nikal kar return kar do
return s.substr(start_index, max_len);
}

int main() {
    string s = "babad";
    cout << "Original String: " << s << endl;
    cout << "Longest Palindrome (Brute): " << longestPalindromeBrute(s) << endl;
    return 0;
}

```

🕒 Time aur Space Complexity

Brute force approach thodi "bhaari" padti hai computer par:

- **Time Complexity:** $O(N^3)$ — Do nested loops lagte hain saari substrings generate karne ke liye ($O(N^2)$), aur har substring ko check karne ke liye hum `isPalindrome` function chalte hain jo $O(N)$ leta hai. Toh total time $(N^2 \times N) = O(N^3)$ ho jata hai. Agar string 1000 characters ki hui, toh yeh code lagbhag 1 arab (1 billion) operations karega, jo interview mein **Time Limit Exceeded (TLE)** de dega.
- **Space Complexity:** $O(1)$ — Humne koi extra string ya space save nahi ki, bas indices (`start_index` aur `max_len`) ka use kiya hai.

(Interview Tip: Jab interviewer bole ki $O(N^3)$ bohot slow hai, tab unhe bolna ki "Sir, isko hum **Expand Around Center** approach se $O(N^2)$ mein aur **Dynamic Programming** se bhi $O(N^2)$ mein optimize kar sakte hain!")

Bhai, $O(N^3)$ wale brute force ko thukra kar $O(N^2)$ par aana hi asli optimization hai!

Longest Palindromic Substring ke liye sabse best aur intuitive optimized approach hai **"Expand Around Center" (Two-Pointer Technique)**. Isme bina kisi extra space ke ($O(1)$ space) kaam ho jata hai, jo ki Dynamic Programming se bhi behtar hai.

Chalo isko पूरी detail mein makkhan ki tarah samajhte hain.

🎯 Core Concept: Expand Around Center Kya Hai?

Brute force mein hum har ek kone se substring bana rahe the. Is approach mein hum ulta sochenge—**hum string ke har ek character ko "Center" (beech ka hissa) maanenge aur wahan se dono taraf (bahar ki taraf) phailenge (Expand karenge).**

Jab tak left aur right wale characters same milte rahenge, hum phailte jayenge. Jaise hi alag character mila, hum ruk jayenge aur wahan tak ki length note kar lenge.

⚠ Do Tarah ke Centers Hote Hain:

Palindromes do tarah ke ho sakte hain, isliye hamare centers bhi do tarah ke honge:

1. **Odd Length Palindrome (e.g., "aba")**: Iska center ek single character hota hai ('b'). Hum left aur right dono ko usi character par rakh kar expand karte hain.
2. **Even Length Palindrome (e.g., "abba")**: Iska center do characters ke beech mein hota hai. Hum left ko pehle 'b' par aur right ko dusre 'b' par rakh kar expand karte hain.

Humein loop chalakar har ek index ke liye yeh dono cases check karne honge.

🔍 Dry Run (Example: "babad")

Hum ek loop chalayenge index $i = 0$ se $n-1$ tak. Maan lo hum index $i = 1$ (character 'a') par hain.

- **Case 1: Odd Length Check (Center = i , i yaani 1, 1)**
- Shuruat: left = 1 ('a'), right = 1 ('a'). Dono same hain? Haan. ("a" mila)
- Expand karo: left = 0 ('b'), right = 2 ('b'). Dono same hain? **Haan**. ("bab" mila)
- Expand karo: left = -1, right = 3 ('a'). left out of bound ho gaya, ruk jao.
- **Max Length mili = 3 ("bab")**
- **Case 2: Even Length Check (Center = i , $i+1$ yaani 1, 2)**
- Shuruat: left = 1 ('a'), right = 2 ('b'). Dono same hain? **Nahi**.
- Loop aage badh jayega.

Aise hi har index ke liye check karte-karte humein sabse badi length mil jayegi.

💻 C++ Code (Optimized)

Yahan iska sabse clean aur effective code hai:

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

// Yeh helper function center se bahar ki taraf expand karega
int expandAroundCenter(string s, int left, int right) {
    int n = s.length();

    // Jab tak characters match kar rahe hain aur hum boundary ke andar hain
    while (left >= 0 && right < n && s[left] == s[right]) {
```

```

        left--; // Peeche jao
        right++; // Aage jao
    }

    // Palindrome ki length return karo
    // (right - left - 1) isliye kyunki loop se bahar aane par left aur right ek
    step extra chal chuke hote hain
    return right - left - 1;
}

string longestPalindrome(string s) {
    int n = s.length();
    if (n < 1) return "";

    int start = 0, max_len = 0;

    for (int i = 0; i < n; i++) {
        // Case 1: Odd length palindrome (Center ek character hai)
        int len1 = expandAroundCenter(s, i, i);

        // Case 2: Even length palindrome (Center do characters ke beech hai)
        int len2 = expandAroundCenter(s, i, i + 1);

        // Dono mein se jo bada ho use chun lo
        int len = max(len1, len2);

        // Agar nayi badi palindrome mili, toh uska starting point aur length
        update karo
        if (len > max_len) {
            max_len = len;
            // Shuruati index nikalne ka formula
            start = i - (len - 1) / 2;
        }
    }

    // Substring cut karke return kar do
    return s.substr(start, max_len);
}

int main() {
    string s = "babad";
    cout << "Original String: " << s << endl;
    cout << "Longest Palindrome (Optimized): " << longestPalindrome(s) << endl;
    return 0;
}

```

Time aur Space Complexity

- **Time Complexity:** $O(N^2)$ — Hum string ke har character (N) par ja rahe hain, aur har character ke liye `expandAroundCenter` function chala rahe hain jo worst-case mein poori string scan kar sakta hai (

$O(N)$). Isliye $N \times N = O(N^2)$. Yeh brute force ($O(N^3)$) se bohot fast hai.

- **Space Complexity:** $O(1)$ — Humne koi matrix ya extra array nahi banaya, bas kuch pointers ka khel hai. Isliye memory bilkul na ke barabar use hoti hai.

🔥 Pro-Tip for Interviews

Agar interviewer aapse kahe ki "**Mujhe isse bhi tez chahiye, kya tum $O(N)$ mein kar sakte ho?**" tab unke saamne bas ek naam le lena: **Manacher's Algorithm**. Yeh ek advance algorithm hai jo isi problem ko linear time $O(N)$ mein solve kar deta hai, halanki interviewers iska code likhne ko bohot kam bolte hain, bas naam hi kaafi hota hai!

Kya aapko yeh logic samajh aaya bhai, ya iske Dynamic Programming ($O(N^2)$ space) wale tarike ko bhi dekhna hai?

Bhai, ab tum seedha DSA ke sabse dhasu aur advanced zone mein kadam rakh rahe ho! Longest Palindromic Substring ko $O(N)$ **Time Complexity** mein solve karne ke liye ek hi king algorithm hai—**Manacher's Algorithm**.

Upar jo humne *Expand Around Center* ($O(N^2)$) padha tha, woh baar-baar ek hi character ko check karta hai. Manacher's Algorithm pehle se check kiye gaye palindromes ki symmetry (mirror property) ka fayda uthakar dubara mehnat karne se bachta hai.

Chalo isko bilkul aasan bhasha mein todkar samajhte hain.

🌀 Core Concept: Iska Dimag Kaise Kaam Karta Hai?

1. Preprocessing (Odd/Even ka jhanjhat khatam)

Pehle waale tarike mein humein Odd aur Even length ke liye alag-alag check karna pad raha tha. Manacher's algorithm isko solve karne ke liye string ke har character ke beech mein, shuruat mein, aur aakhiri mein ek special character (jaise #) daal deta hai.

- Agar string hai "aba", toh woh ban jayegi "#a#b#a#".
- Ab string ki length hamesha **Odd** ho jayegi, toh hamein do alag cases check nahi karne padenge.

2. Mirror Property (Asli Magic ✨)

Maan lo hum ek bada palindrome dhoodh chuke hain jiska center C hai aur uski right boundary R tak jaati hai.

Agar hum is boundary ke andar kisi naye index i par hain, toh humein zero se expand karne ki zaroorat nahi hai! Hum i ka **Mirror Index** (i') nikalenge jo center C ke peeche hoga ($i' = 2C - i$). Kyunki center ke aas-pas sab mirror hota hai, toh jo palindrome i' par bana tha, waisa hi palindrome i par bhi banega! Hum wahan se seedha copy kar lete hain aur sirf tab aage expand karte hain jab boundary R ke baahar nikalna ho.

💻 C++ Code (Manacher's Algorithm)

Yeh code dekhne mein thoda bada lag sakta hai, par iska ek-ek step upar bataye logic par hi tika hai:

```

#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

string manachersAlgorithm(string s) {
    // Step 1: Preprocess string (e.g., "aba" -> "#a#b#a#")
    string T = "#";
    for (char c : s) {
        T += c;
        T += "#";
    }

    int n = T.length();
    // P array store karega har index par banne wale palindrome ka radius
    vector<int> P(n, 0);

    int C = 0; // Current Palindrome ka Center
    int R = 0; // Current Palindrome ki Right Boundary

    for (int i = 0; i < n; i++) {
        // i' (i-prime) nikal rahe hain jo i ka mirror hai across Center C
        int i_mirror = 2 * C - i;

        // Agar i boundary R ke andar hai, toh purana data copy karo
        if (i < R) {
            P[i] = min(R - i, P[i_mirror]);
        }

        // Jo bacha hua part hai, uske liye bahar ki taraf expand karo
        while (i - 1 - P[i] >= 0 && i + 1 + P[i] < n && T[i - 1 - P[i]] == T[i + 1
+ P[i]]) {
            P[i]++;
        }

        // Agar naya palindrome boundary R ke baahar nikal gaya, toh Center aur
        Boundary badal do
        if (i + P[i] > R) {
            C = i;
            R = i + P[i];
        }
    }

    // P array mein se sabse bada radius aur uska center dhoodho
    int max_len = 0;
    int center_index = 0;
    for (int i = 0; i < n; i++) {
        if (P[i] > max_len) {
            max_len = P[i];
            center_index = i;
        }
    }
}

```

```

    }

    // Preprocessed string se original string ka answer nikalna
    int start = (center_index - max_len) / 2;
    return s.substr(start, max_len);
}

int main() {
    string s = "babad";
    cout << "Original String: " << s << endl;
    cout << "Longest Palindrome ($O(N)$): " << manachersAlgorithm(s) << endl;
    return 0;
}

```

🕒 Time aur Space Complexity

- **Time Complexity:** $O(N)$ — Yeh is algorithm ki sabse badi khasiyat hai. Aapko loop ke andar **while** loop dikh raha hoga, par woh **while** loop tabhi chalta hai jab hum boundary R ko aage badhate hain. Puri string mein boundary R sirf N baar hi aage badh sakti hai, isliye total time linear ($O(N)$) hota hai.
- **Space Complexity:** $O(N)$ — Humne string ko preprocess karne ke liye ek nayi string T aur radius store karne ke liye ek array P banaya hai, jo $O(N)$ memory leta hai.

💡 Interview Me Kab Batana Hai?

- Jab interviewer kahe ki mujhe $O(N^2)$ **se bhi tez** solution chahiye.
- Iska code bohot rare pucha jata hai kyunki yeh kaafi bada hai, lekin agar aapne iska **Symmetry/Mirror waala concept** interviewer ko samjha diya, toh unka dil khush ho jayega aur aapka selection pakka samajho bhai!

Bhai, coding interviews mein code likhne se pehle khud se **Edge Cases** batana interviewer ke saamne aapko ek pro coder dikhata hai. Isse pata chalta hai ki aap sirf code rat ke nahi aaye ho, balki har situation ke baare mein soch rahe ho.

Longest Palindromic Substring (LPS) ke saare important edge cases yeh rahe, unke inputs, outputs aur reasons ke saath:

1. Empty String (Khaali String)

- **Input:** $s = ""$
- **Expected Output:** $""$
- **Kyun zaroori hai?** Agar aapne code ki shuruat mein hi length check nahi ki, toh aapka code garbage value utha sakta hai ya loop chalane par **Segmentation Fault (Runtime Error)** de sakta hai.
- **Fix:** Code ke starting mein hi handle karo: `if(s.length() == 0) return "";`

2. Single Character String (Sirf Ek Akshar)

- **Input:** `s = "a"`
- **Expected Output:** `"a"`
- **Kyun zaroori hai?** Har akshar khud mein ek palindrome hota hai jiski length 1 hoti hai. Aapka loop is single character ko sahi se handle karke wahi return karna chahiye, out-of-bound nahi jaana chahiye.

3. Saare Characters Same Hon (All Identical Characters)

- **Input:** `s = "aaaaa"`
- **Expected Output:** `"aaaaa"`
- **Kyun zaroori hai?** Is case mein poori ki poori string hi longest palindrome hai. Jab aap center se expand karoge, toh pointers bilkul extreme corners tak stretch hone chahiye bina crash kiye.

4. Koyi Bhi Palindrome Na Ho (All Distinct Characters)

- **Input:** `s = "abcdef"`
- **Expected Output:** `"a"` (ya koi bhi ek single character jaise `"b"`, `"c"`, kyunki sabki max length 1 hi hai)
- **Kyun zaroori hai?** Isme koi bhi do akshar milkar palindrome nahi bana rahe. Aapke code ka default `max_len` hamesha `1` par set hona chahiye, taaki aisa case aane par woh zero ya khali string return na kare, balki pehla character return kar de.

5. Even vs Odd Length Palindromes

- **Odd Length Input:** `s = "aba" → Output: "aba"` (Center ek single character `'b'` hai)
- **Even Length Input:** `s = "abba" → Output: "abba"` (Center do characters ke beech ka gap hai)
- **Kyun zaroori hai?** Agar aapne *Expand Around Center* approach mein sirf `expand(s, i, i)` chalaya aur `expand(s, i, i+1)` chalana bhool gaye, toh aapka code `"abba"` waale even cases ko kabhi pakad hi nahi payega aur galat answer dega.

6. Multiple Palindromes of Same Maximum Length

- **Input:** `s = "babad"`
- **Expected Output:** `"bab"` ya `"aba"` (Dono ki length 3 hai)
- **Input 2:** `s = "aacdeeff"` → **Expected Output:** `"aa"` (Yahan `"aa"`, `"ee"`, `"ff"` sabki length 2 hai)
- **Kyun zaroori hai?** Standard rule yeh hota hai ki jo palindrome **sabse pehle** aata hai, hum use return karte hain. Iske liye aapka logic strictly bada hona chahiye: `if (current_len > max_len)` tabhi update karo. Agar aapne `>=` laga diya, toh woh aakhiri waala utha lega, jo aksar galat mana jata hai.

7. Palindrome Bilkul Kno par Ho (At Extreme Boundaries)

- **Left Corner Par:** `s = "racecarxyz" → Output: "racecar"`
- **Right Corner Par:** `s = "xyzracecar" → Output: "racecar"`
- **Kyun zaroori hai?** Isse aapke pointers ki boundary condition test hoti hai. Jab `left` ghat-ghat kar `0` par pahuche ya `right` badh-badh kar `n-1` par pahuche, toh loop smoothly rukna chahiye (`left >= 0 && right < n`).

Interview Tip

Jab interviewer aapse bole *"Write the code for Longest Palindromic Substring"*, toh code likhne se pehle whiteboard ya notepad par yeh likhna:

"Sir, before diving into the code, let me clarify the constraints and edge cases. What should be returned if the string is empty? Should I return the first occurrence if there are multiple palindromes of the same length? And I'll make sure to handle both even and odd length palindromes."

Yeh sunte hi interviewer samajh jayega ki bande ka logic aur dimaag dono bohot solid hain!

Bhai, coding interviews mein code likhne se pehle khud se **Edge Cases** batana interviewer ke saamne aapko ek pro coder dikhata hai. Isse pata chalta hai ki aap sirf code rat ke nahi aaye ho, balki har situation ke baare mein soch rahe ho.

Longest Palindromic Substring (LPS) ke saare important edge cases yeh rahe, unke inputs, outputs aur reasons ke saath:

1. Empty String (Khaali String)

- **Input:** `s = ""`
- **Expected Output:** `""`
- **Kyun zaroori hai?** Agar aapne code ki shuruat mein hi length check nahi ki, toh aapka code garbage value utha sakta hai ya loop chalane par **Segmentation Fault (Runtime Error)** de sakta hai.
- **Fix:** Code ke starting mein hi handle karo: `if(s.length() == 0) return "";`

2. Single Character String (Sirf Ek Akshar)

- **Input:** `s = "a"`
- **Expected Output:** `"a"`
- **Kyun zaroori hai?** Har akshar khud mein ek palindrome hota hai jiski length 1 hoti hai. Aapka loop is single character ko sahi se handle karke wahi return karna chahiye, out-of-bound nahi jaana chahiye.

3. Saare Characters Same Hon (All Identical Characters)

- **Input:** `s = "aaaaa"`
- **Expected Output:** `"aaaaa"`
- **Kyun zaroori hai?** Is case mein poori ki poori string hi longest palindrome hai. Jab aap center se expand karoge, toh pointers bilkul extreme corners tak stretch hone chahiye bina crash kiye.

4. Koyi Bhi Palindrome Na Ho (All Distinct Characters)

- **Input:** `s = "abcdef"`
- **Expected Output:** `"a"` (ya koi bhi ek single character jaise `"b"`, `"c"`, kyunki sabki max length 1 hi hai)
- **Kyun zaroori hai?** Isme koi bhi do akshar milkar palindrome nahi bana rahe. Aapke code ka default `max_len` hamesha `1` par set hona chahiye, taaki aisa case aane par woh zero ya khali string return na kare, balki pehla character return kar de.

5. Even vs Odd Length Palindromes

- **Odd Length Input:** `s = "aba" → Output: "aba"` (Center ek single character `'b'` hai)
- **Even Length Input:** `s = "abba" → Output: "abba"` (Center do characters ke beech ka gap hai)
- **Kyun zaroori hai?** Agar aapne *Expand Around Center* approach mein sirf `expand(s, i, i)` chalaya aur `expand(s, i, i+1)` chalana bhool gaye, toh aapka code `"abba"` waale even cases ko kabhi pakad hi

nahi payega aur galat answer dega.

6. Multiple Palindromes of Same Maximum Length

- **Input:** `s = "babad"`
- **Expected Output:** `"bab"` ya `"aba"` (Dono ki length 3 hai)
- **Input 2:** `s = "aacdeeff"` → **Expected Output:** `"aa"` (Yahan `"aa"`, `"ee"`, `"ff"` sabki length 2 hai)
- **Kyun zaroori hai?** Standard rule yeh hota hai ki jo palindrome **sabse pehle** aata hai, hum use return karte hain. Iske liye aapka logic strictly bada hona chahiye: `if (current_len > max_len)` tabhi update karo. Agar aapne `>=` laga diya, toh woh aakhiri waala utha lega, jo aksar galat mana jata hai.

7. Palindrome Bilkul Kno par Ho (At Extreme Boundaries)

- **Left Corner Par:** `s = "racecarxyz"` → **Output:** `"racecar"`
- **Right Corner Par:** `s = "xyzracecar"` → **Output:** `"racecar"`
- **Kyun zaroori hai?** Isse aapke pointers ki boundary condition test hoti hai. Jab `left` ghat-ghat kar `0` par pahuche ya `right` badh-badh kar `n-1` par pahuche, toh loop smoothly rukna chahiye (`left >= 0 && right < n`).

Interview Tip

Jab interviewer aapse bole *"Write the code for Longest Palindromic Substring"*, toh code likhne se pehle whiteboard ya notepad par yeh likhna:

"Sir, before diving into the code, let me clarify the constraints and edge cases. What should be returned if the string is empty? Should I return the first occurrence if there are multiple palindromes of the same length? And I'll make sure to handle both even and odd length palindromes."

Yeh sunte hi interviewer samajh jayega ki bande ka logic aur dimaag dono bohot solid hain!

Bhai, brute force mein hum baar-baar loop chala kar sahi word dhoond rahe the. Ab seekhte hain iska sabse khatarnak aur optimized tarika, jisme sirf **ek baar** sentence ko padhna padega aur kaam khatam!

Is approach ko hum **Direct Mapping** ya **Bucket Sort** ka logic bhi keh sakte hain.

Core Concept: Direct Indexing

Humein pata hai ki sentence mein max 9 words ho sakte hain aur har word ke aakhiri mein uski sahi position (`1` se `9`) likhi hui hai.

Toh baar-baar dhoondne ke bajaye, hum **9 size ka ek khali vector (buckets)** bana lenge. Jaise hi humein koi word milega, hum uska number dekhenge aur use **seedha us number waale index par patak (insert kar) denge**.

Step-by-Step Logic:

1. 10 size ka ek string vector banao: `vector<string> temp(10, "")`. (10 size isliye taaki `1` se `9` tak ke indices safe rahein, `0` waale ko khali chhod denge).
2. Sentence se ek-ek karke word nikalo.

3. Har word ka aakhiri character nikal kar pata karo ki uska index kya hai (jaise "This1" ka index 1 hai).
4. Us word ke aakhiri se number ko hata do (`pop_back()`).
5. Vector mein us index par word ko store kar do: `temp[index] = word`.
6. Aakhiri mein, `temp` vector par 1 se lekar aakhiri tak ek simple loop chalao aur saare words ko ek saath space dekar jod do.

Dry Run (Makkhan ki tarah samjho)

Input: "is2 sentence4 This1 a3" Hamara `temp` vector shuruat mein khali hai: [, , , , , ...]

- **Word 1: "is2"**
- Index nikal nikal: 2
- Number hatane ke baad word: "is"
- `temp[2] = "is"` → Vector bana: [, , "is", ,]
- **Word 2: "sentence4"**
- Index: 4, Word: "sentence"
- `temp[4] = "sentence"` → Vector bana: [, , "is", , "sentence"]
- **Word 3: "This1"**
- Index: 1, Word: "This"
- `temp[1] = "This"` → Vector bana: [, "This", "is", , "sentence"]
- **Word 4: "a3"**
- Index: 3, Word: "a"
- `temp[3] = "a"` → Vector bana: [, "This", "is", "a", "sentence"]

Ab bas vector ke elements ko 1 se 4 tak uthao aur jod do: "This is a sentence".

C++ Code (Optimized $O(N)$)

Yahan iska sabse efficient C++ code hai:

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
using namespace std;

string sortSentenceOptimized(string s) {
    // 10 size ka vector liya kyunki numbers 1 se 9 tak hi ho sakte hain
    vector<string> temp(10, "");
```

```

stringstream ss(s);
string word;
int word_count = 0;

// Sentence se ek-ek word nikal rahe hain (Single Pass)
while (ss >> word) {
    // Word ka aakhiri character nikal kar integer mein badla
    int index = word.back() - '0';

    word.pop_back(); // Word ke aakhiri se number hata diya

    temp[index] = word; // Sahi index par word ko direct bitha diya
    word_count++; // Total kitne words hain track rakhne ke liye
}

string ans = "";
// 1 se lekar jitne words the, unhe line se jod do
for (int i = 1; i <= word_count; i++) {
    ans += temp[i];
    if (i != word_count) {
        ans += " "; // Aakhiri word ko chhodkar sabke baad space lagao
    }
}

return ans;
}

int main() {
    string s = "is2 sentence4 This1 a3";
    cout << "Original: " << s << endl;
    cout << "Optimized Sorted: " << sortSentenceOptimized(s) << endl;
    return 0;
}

```

Time aur Space Complexity

- **Time Complexity:** $O(N)$ — Yahan N string ki total length hai. Humne sentence ke har ek character ko sirf ek baar scan kiya hai words alag karne ke liye. Uske baad jo loop chal raha hai woh hamesha fix (max 9 baar) chalega, jo ki constant $O(1)$ hai. Isliye total time linear ($O(N)$) ho gaya.
- **Space Complexity:** $O(N)$ — Humne `temp` vector banaya hai jiski size fix 10 hai (jo ki $O(1)$ space hai), par jo final `ans` string aur tokens ban rahe hain woh original sentence ke barabar memory lenge, isliye total space complexity $O(N)$ hogi.

Yeh solution solid hai bhai! Interviewer ko brute force ke baad seedha yeh batana, woh khush ho jayega.

Bhai, tumne bilkul pro-coder waali baat ki hai! $O(N)$ Time Complexity se tez toh nahi ho sakta (kyunki sentence ko ek baar padhna hi padega), lekin hum **Memory (Space) aur Speed ke internal overhead ko aur zyada optimize** kar sakte hain.

Pichle code mein hum `stringstream` aur `vector<string>` use kar rahe تھے۔ Yeh dono background mein naye string objects banate hain, jisse **Heap Memory Allocation** hoti hai, jo code ko thoda slow bana deti hai.

Hum ise **Zero-Copy / Index Mapping** se aur optimize karenge, jahan hum koi naya string object ya vector nahi banayenge. Hum bas original string ke andar hi words ke **indices (starting position aur length)** ko yaad rakhenge.

🧠 Ultimate Optimization Logic

Hum 10 size ka ek fixed array banayenge, par strings ka nahi, balki **integers (pairs)** ka. `pair<int, int> word_positions[10];` → Yeh store karega `{Word_Start_Index, Word_Length}`.

1. Original string par ek single loop chalao.
2. Jaise hi ek word khatam ho (space mile ya string khatam ho), uske aakhiri character se number (index) nikal lo.
3. Us word ka starting point aur uski length (number ko chhod kar) hamare fixed array mein store kar do.
4. Aakhiri mein, ek nayi string `ans` banao (jo hamein return karni hi hai) aur use `reserve()` kar do taaki baar-baar memory resize na ho. Array se positions uthao aur original string se direct characters copy kar do.

Isse poore code mein **ek bhi faltu string copy nahi hoti!**

📄 Highly Optimized C++ Code

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

string sortSentenceHighlyOptimized(string s) {
    int n = s.length();

    // Fixed array: Jo har word ka {start_index, length} store karega
    // Size 10 li hai kyunki words 1 se 9 tak hi ho sakte hain
    pair<int, int> word_map[10];

    int start = 0;
    int word_count = 0;

    // Single pass loop: Pura sentence ek baar scan ho raha hai
    for (int i = 0; i <= n; i++) {
        // Agar space mila ya string khatam hui, matlab ek word complete hua
        if (i == n || s[i] == ' ') {
            int word_end = i - 1; // Word ka aakhiri character (jo ki ek number
            hai)

            int digit = s[word_end] - '0'; // Char to Int conversion

            int word_len = word_end - start; // Number ko hata kar word ki length
```

```

        // Map mein store kar liya: digit ke respect mein {shuruat, lambai}
        word_map[digit] = {start, word_len};

        word_count++;
        start = i + 1; // Agle word ka starting point
    }
}

// Answer string banayi aur use pehle hi exact size de di (Memory
Optimization)
string ans = "";
ans.reserve(n);

// Array se direct position utha kar answer build karo
for (int i = 1; i <= word_count; i++) {
    int w_start = word_map[i].first;
    int w_len = word_map[i].second;

    // original string 's' se direct substring utha kar jodo (No extra
    allocations)
    ans.append(s, w_start, w_len);

    if (i != word_count) {
        ans += " "; // Aakhiri word ko chhod kar space lagao
    }
}

return ans;
}

int main() {
    string s = "is2 sentence4 This1 a3";
    cout << "Original: " << s << endl;
    cout << "Highly Optimized: " << sortSentenceHighlyOptimized(s) << endl;
    return 0;
}

```

🕒 Yeh Pehle Se Behtar Kyun Hai?

Feature	Pichla Optimized Solution	Yeh Waala Highly Optimized Solution
Time Complexity	$O(N)$	$O(N)$ (Lekin actual execution time aur kam ho jayega)
Auxiliary Space	$O(N)$ (Kyunki <code>vector<string></code> mein naye words ban rahe the)	$O(1)$ (Kyunki humne sirf 10 pairs ka fixed array banaya hai, koi naye words nahi banaye)
Heap Allocations	High (Kyunki <code>stringstream</code> aur <code>vector<string></code> memory allocate karte)	Zero (Sirf ek baar final answer string ke liye allocation hoti hai)

Feature	Pichla Optimized Solution	Yeh Waala Highly Optimized Solution
	hain)	
Bhai, yeh C++ ka sabse fast aur memory-efficient tarika hai. Interviewer agar memory constraints par push kare, toh yeh solution unhe hila dega!		
Bhai, " Sort Vowels in a String " bhi ek bohot hi mast question hai. Isme aksar log galti yeh karte hain ki poori string ko sort karne lagte hain, jabki hamein consonants (baki aksharon) ko chhedna bhi nahi hai!		
Chalo iska sabse basic aur aasan Brute Force tarika poori detail mein step-by-step samajhte hain.		

Problem Kya Hai?

Aapko ek string di jayegi. Aapko us string ke andar jitne bhi **vowels** (A, E, I, O, U aur a, e, i, o, u) hain, unhe unke ASCII values ke hisaab se alphabetical order mein sort karna hai, lekin baaki saare characters (consonants) apni hi jagah par rehne chahiye.

- **Input:** "lEetcOde"
- **Output:** "lEotcede"

 **Dhyan dene waali baat:** ASCII values mein saare Uppercase letters (A = 65) Lowercase letters (a = 97) se chhote hote hain. Isliye sorting ka order hamesha aisa hoga: A -> E -> I -> O -> U -> a -> e -> i -> o -> u.

Brute Force Logic (3 Steps Concept)

Brute force ka seedha funda hai—kaam ko alag-alag tukdon mein baant do:

1. **Step 1 (Extract):** Poori string par ek loop chalayein. Jahan bhi koi vowel dikhe, use utha kar ek alag list ya string (`vowels_list`) mein store kar lein.
2. **Step 2 (Brute Sort):** Ab us `vowels_list` ko nested loops (Bubble ya Selection Sort) ka use karke sort karein.
3. **Step 3 (Place Back):** Original string par fir se loop chalayein. Jaise hi koi vowel mile, use hamari sorted `vowels_list` ke pehle character se replace kar dein, aur list mein aage badh jayein.

Dry Run (Makkhan ki tarah samjho)

Maan lo input hai: "lEetcOde"

- **Step 1: Vowels nikalna**
 - Scan karte hain: 'E' mila, 'e' mila, 'O' mila, 'e' mila.
 - Hamari `vowels_list` bani: ['E', 'e', 'O', 'e']
- **Step 2: Brute Sort karna (ASCII ke hisaab se)**
 - Nested loops chalakar jab hum ['E', 'e', 'O', 'e'] ko sort karenge:

- Kyunki 'O' (79) chhota hai 'e' (101) se, toh yeh sort hokar banega: ['E', 'O', 'e', 'e']
- **Step 3: Wapas string mein bithaana**
- Original string: l [E] e t c [O] d [e]
- Pehla vowel mila E → Sorted ka 1st element E lagaya → "lE..."
- Dusra vowel mila e → Sorted ka 2nd element O lagaya → "lEo..."
- Tisra vowel mila O → Sorted ka 3rd element e lagaya → "lEotce..."
- Chotha vowel mila e → Sorted ka 4th element e lagaya → "lEotcede"

C++ Code (Pure Brute Force)

Yahan bina kisi built-in `sort()` function ke, bilkul desi loops se likha hua code hai:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Helper function check karne ke liye ki character vowel hai ya nahi
bool isVowel(char ch) {
    return (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U');
}

string sortVowelsBrute(string s) {
    vector<char> vowels;

    // Step 1: Saare vowels ko alag vector mein nikal lo
    for (char ch : s) {
        if (isVowel(ch)) {
            vowels.push_back(ch);
        }
    }

    // Step 2: Vowels wale vector ko brute force sort (Bubble Sort) karo
    int num_vowels = vowels.size();
    for (int i = 0; i < num_vowels - 1; i++) {
        for (int j = i + 1; j < num_vowels; j++) {
            // ASCII value ke hisaab se swap kar rahe hain
            if (vowels[i] > vowels[j]) {
                char temp = vowels[i];
                vowels[i] = vowels[j];
                vowels[j] = temp;
            }
        }
    }
}
```

```
// Step 3: Sorted vowels ko wapas original string mein bithaao
int vowel_index = 0;
for (int i = 0; i < s.length(); i++) {
    if (isVowel(s[i])) {
        s[i] = vowels[vowel_index]; // Replace kiya
        vowel_index++; // Agle sorted vowel par jao
    }
}

return s;
}

int main() {
    string str = "lEetcOde";
    cout << "Original: " << str << endl;
    cout << "Sorted Vowels: " << sortVowelsBrute(str) << endl;
    return 0;
}
```

🕒 Time aur Space Complexity

- **Time Complexity:** $O(N + V^2)$ — Yahan N string ki length hai (extract aur place karne ke liye). V string mein मौजूद vowels ki sankhya hai. Kyunki humne vowels ko sort karne ke liye nested loops chalayе hain, isliye $O(V^2)$ time lagega. Worst case mein agar saare characters vowels hon (jaise "aeiou"), toh yeh $O(N^2)$ ban jayega.
- **Space Complexity:** $O(V)$ — Humne vowels ko alag se store karne ke liye ek vector banaya hai, jo worst case mein string ki length (N) ke barabar space le sakta hai.

(Interview Tip: Agar interviewer kahe ki sorting ko aur tez karo, toh bolna ki "Sir, Step 2 mein brute force loops ki jagah hum standard `sort()` function laga kar $O(V \log V)$ kar sakte hain, ya fir **Counting Sort** ka use karke bina kisi comparison ke pure $O(N)$ mein bhi vowels ko sort kiya ja sakta hai!")

Bhai, pichle brute force approach mein humne vowels ko alag nikala aur phir unhe sort kiya, jisme $O(V^2)$ ya $O(V \log V)$ ka time lag raha tha.

Lekin ab seekhte hain iska **sabse khatarnak aur fully optimized tarika**, jo bina kisi comparison ke pure $O(N)$ **Time Complexity** aur $O(1)$ **Extra Space** mein solve ho jata hai. Isme hum use karenge **Counting Sort** ka logic!

🎯 Core Concept: Counting Sort (No Sorting Needed!)

Humein pata hai ki English mein vowels bilkul fixed hain—sirf **10 vowels** hote hain (5 lowercase aur 5 uppercase). Aur unka aapas mein sahi sorting order (ASCII ke hisaab se) hamesha fix rahega: "AEIOUaeiou"

Toh kyun na hum vowels ko sort karne ke bajaye, sirf unki ginti (frequency) yaad rakh lein?

Step-by-Step Logic:

1. Ek frequency array ya count array banao (size 256 ka, taaki har character ki ASCII value fit ho sake) aur sabme 0 bhar do.
2. **First Pass (Ginti karo):** Original string par ek baar loop chalao. Jahan bhi koi vowel mile, uske count ko array mein badha do (`count[ch]++`).
3. **Second Pass (Wapas bithaao):** Ab original string par dobara loop chalao. Saath hi, ek pointer rakho jo hamare fixed sorted order ("AEIOUaeiou") par nazar rakhega. Jaise hi string mein koi vowel mile, use us character se replace kar do jiska count abhi 0 se bada hai, aur uska count ek kam kar do.

Dry Run (Makkhan ki tarah samjho)

Input: "lEetC0de" Humara fixed order hai: "AEIOUaeiou"

- **Pass 1: Frequency Count** String scan karne par mila: E (1 baar), e (2 baar), 0 (1 baar). Hamare paas count taiyar hai: `count['E']=1, count['0']=1, count['e']=2`. Baaki sab 0 hain.
- **Pass 2: Re-substituting (In-Place)** Hum string par dobara challenge aur fixed order "AEIOUaeiou" ke hisaab se characters uthayenge:
 - Pehla vowel mila E → Fixed order mein check kiya, `count['E']` hai 1 → Replace kiya E se, count hua 0. String: "lE..."
 - Dusra vowel mila e → Fixed order mein E ab 0 hai. Agla non-zero mila 0 (`count['0']=1`) → Replace kiya 0 se, count hua 0. String: "lEo..."
 - Tisra vowel mila 0 → Fixed order mein E aur 0 khatam. Agla non-zero mila e (`count['e']=2`) → Replace kiya e se, count hua 1. String: "lEotce..."
 - Chotha vowel mila e → `count['e']` abhi bhi 1 hai → Replace kiya e se, count hua 0. String: "lEotcede"

C++ Code (Highly Optimized $O(N)$)

Yahan iska sabse efficient in-place solution hai, jisme humne koi extra string ya vector create nahi kiya:

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// Helper function check karne ke liye ki character vowel hai ya nahi
bool isVowel(char ch) {
    return (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U');
}

string sortVowelsOptimized(string s) {
    // 256 size ka array liya taaki kisi bhi character ki ASCII value direct index
    // ban sake
    int count[256] = {0};

    // 1. Saare vowels ki frequency count kar lo (Single Pass)
    for (char ch : s) {
        if (isVowel(ch)) {
```

```

        count[ch]++;
    }
}

// Vowels ka sahi sorted order ASCII ke hisaab se
string sorted_order = "AEIOUaeiou";
int vowel_ptr = 0; // Yeh pointer 'sorted_order' string par chalega

// 2. Original string mein in-place values replace karo
for (int i = 0; i < s.length(); i++) {
    if (isVowel(s[i])) {

        // Jab tak current vowel ka count 0 hai, agle sorted vowel par jao
        while (count[sorted_order[vowel_ptr]] == 0) {
            vowel_ptr++;
        }

        // Sahi sorted vowel ko apni jagah bithaao
        s[i] = sorted_order[vowel_ptr];

        // Use karne ke baad uska count ek kam kar do
        count[sorted_order[vowel_ptr]]--;
    }
}

return s;
}

int main() {
    string str = "lEetC0de";
    cout << "Original: " << str << endl;
    cout << "Optimized Sorted Vowels: " << sortVowelsOptimized(str) << endl;
    return 0;
}

```

🕒 Time aur Space Complexity

- **Time Complexity:** $O(N)$ — Humne string ko sirf do baar scan kiya (ek baar count karne ke liye aur ek baar replace karne ke liye). Jo andar waala **while** loop hai, woh max 10 baar hi chal sakta hai poore program mein (kyunki **vowels_ptr** sirf 0 se 9 tak hi badhega). Isliye total time pure linear ($O(N)$) hai.
- **Space Complexity:** $O(1)$ — Humne sirf 256 size ka ek fixed integer array banaya hai, jo ki constant space leta hai. Humne koi naya vector ya alag se vowels store karne ke liye dynamic memory use nahi ki. Jo bhi badlav kiye, original string **s** ke andar hi kiye (In-place execution).

Bhai, yeh is problem ka absolute best solution hai. Interview mein brute force batane ke baad jab tum yeh counting sort waala logic doge, toh interviewer samajh jayega ki aapka basic core computer science aur memory management dono solid hain!

Bhai, agar tum interview mein built-in `std::sort()` use karke `vector` ke saath is problem ko solve karna chahte ho, toh yeh approach brute force se kaafi behtar aur samajhne mein sabse aasan hai.

Isme hum ek `vector<char>` banayenge, saare vowels usme daalenge, use standard sorting algorithm se sort karenge, aur wapas string mein fit kar denge.

Vector Approach Ka Logic

Isko hum **3 simple steps** mein karenge:

1. **Collect:** Puri string par loop chalao. Jo bhi vowel mile, use `vector<char> vowels` mein `push_back()` karte jao.
2. **Sort:** C++ ke built-in `sort(vowels.begin(), vowels.end())` ka use karke us vector ko sort kar lo. Yeh background mein **IntroSort** use karta hai jo bohot fast hota hai.
3. **Replace:** Original string par dobara loop chalao. Jahan bhi vowel dikhe, use sorted vector ke elements se ek-ek karke badalte jao.

C++ Code (Using Vector & `std::sort`)

Yahan iska simple aur standard code hai:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm> // std::sort use karne ke liye
using namespace std;

// Helper function check karne ke liye ki character vowel hai ya nahi
bool isVowel(char ch) {
    return (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U');
}

string sortVowelsWithVector(string s) {
    vector<char> vowels;

    // Step 1: Saare vowels ko nikal kar vector mein bharo
    for (char ch : s) {
        if (isVowel(ch)) {
            vowels.push_back(ch);
        }
    }

    // Step 2: Vector ko built-in sort function se sort karo
    // Yeh O(V log V) time lega, jahan V = number of vowels hai
    sort(vowels.begin(), vowels.end());

    // Step 3: Sorted vowels ko wapas original string mein bithaao
    int ptr = 0; // Yeh pointer vector ke elements par track rakhega
    for (int i = 0; i < s.length(); i++) {
```



```

        if (isVowel(s[i])) {
            s[i] = vowels[ptr]; // Original character ko sorted vowel se badla
            ptr++; // Vector mein agle element par gaye
        }
    }

    return s;
}

int main() {
    string str = "lEetC0de";
    cout << "Original: " << str << endl;
    cout << "Sorted using Vector: " << sortVowelsWithVector(str) << endl;
    return 0;
}

```

🕒 Time aur Space Complexity

Yeh approach brute force se kaafi optimize hai, par frequency array (counting sort) se thodi peeche hai:

- **Time Complexity:** $O(N + V \log V)$ — Jahan N string ki total length hai aur V string mein मौजूद vowels ki sankhya hai. String ko scan karne mein $O(N)$ laga aur vector ko sort karne mein $O(V \log V)$ laga. Yeh brute force ke $O(V^2)$ se lakho guna behtar hai.
- **Space Complexity:** $O(V)$ — Humne vowels ko store karne ke liye ek alag vector banaya hai. Worst case mein agar poori string hi vowels ki hui (jaise "aeiou"), toh yeh space $O(N)$ tak ja sakti hai.

💡 Interview Ke Liye Tip

Agar interview mein Counting Sort ($O(N)$) dimaag mein na aaye, toh sabse pehle yeh `**Vector + std::sort**` waala tarika hi batana chahiye. Yeh likhne mein chota hota hai aur isme galti hone ke chance na ke barabar hote hain!

Bhai, LeetCode ka **"Add Strings"** ek aisa classic question hai jo dekhne mein bohot aasan lagta hai, par interviewer constraints lagakar iska maza badha deta hai. Constraints kya hote hain? Aap in strings ko direct integer mein convert nahi kar sakte (kyunki numbers इतने bade ho sakte hain jo `long long int` mein bhi fit na hon), aur aapko koi BigInteger library bhi use nahi karni hai.

Iska matlab, humein bilkul **bachpan waala primary school ka maths** simulation code mein likhna padega! Chalo iski intuition, optimized approach, aur dry run ko makkhan ki tarah samajhte hain.

💡 Intuition: Bachpan Ka Maths 📝

Jab humein paper par 456 aur 77 ko add karna hota tha, toh hum kya karte the?

1. Hum dono numbers ko right side (ones place) se align karte the.
2. Sabse aakhiri digits ko uthate the: $6 + 7 = 13$.
3. 3 ko neeche likhte the aur 1 ko aage waale column par **Carry** bhej dete the.
4. Fir agle digits aur carry ko add karte the.

Humein code mein bhi yahi karna hai! Hum do pointers lenge—ek pehli string ke aakhiri mein aur dusra dusri string ke aakhiri mein—aur peeche se aage ki taraf badhenge.

🚀 Optimized Approach: Single-Pass Simulation

Is problem ka sabse optimized tarika yahi hai ki hum bina kisi extra conversion ke, single loop ke andar character-by-character addition karte jayein.

Algorithm Steps:

1. Do pointers banao: `i` (`num1` ke end par) aur `j` (`num2` ke end par).
 2. Ek variable lo `carry = 0`.
 3. Ek loop chalao jo tab tak chalega jab tak `i >= 0` **YA** `j >= 0` **YA** `carry > 0` ho. (Yaani jab tak digits bache hain ya carry bacha hai).
 4. Har baar dono strings se current character uthao, use integer mein badlo (by subtracting `'0'`). Agar pointer string ke bahar nikal gaya ho, toh uski value `0` maan lo.
 5. `sum = digit1 + digit2 + carry` nikal lo.
 6. Naya `carry` nikalne ka formula: `carry = sum / 10`.
 7. Jo digit humein answer mein jodna hai, uska formula: `sum % 10`.
 8. Aakhiri mein hamara answer ulta (reversed) banega, toh hum string ko `reverse()` karke sahi answer return kar denge.
-

🔍 Dry Run (Example: `num1 = "456"`, `num2 = "77"`)

- Initial State: `i = 2` (points to '6'), `j = 1` (points to '7'), `carry = 0`, `ans = ""`

Iteration 1:

- `num1` se mila 6, `num2` se mila 7.
- `sum = 6 + 7 + 0 (carry) = 13`
- `carry = 13 / 10 = 1`
- Current Digit = `13 % 10 = 3`
- `ans = "3"`
- Pointers update: `i = 1, j = 0`

Iteration 2:

- `num1` se mila 5, `num2` se mila 7.
- `sum = 5 + 7 + 1 (carry) = 13`
- `carry = 13 / 10 = 1`
- Current Digit = `13 % 10 = 3`
- `ans = "33"`
- Pointers update: `i = 0, j = -1` (string 2 khatam!)

Iteration 3:

- `num1` se mila 4, `num2` out of bound ho gaya toh mila 0.

- $sum = 4 + 0 + 1$ (carry) = 5
- $carry = 5 / 10 = 0$
- Current Digit = $5 \% 10 = 5$
- `ans = "335"`
- Pointers update: $i = -1, j = -1$

Loop End: Kyunki `i, j` dono khatam ho gaye aur `carry` bhi 0 ho gaya.

- Ab `ans` ko reverse kar do: `"335"` ban gaya `"533"`. (Bilkul sahi answer!)

C++ Code (Most Optimized Version)

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

string addStrings(string num1, string num2) {
    string ans = "";

    // Dono strings ke aakhiri index par pointers set kiye
    int i = num1.length() - 1;
    int j = num2.length() - 1;
    int carry = 0;

    // Jab tak koi bhi ek condition sach hai, loop chalega
    while (i >= 0 || j >= 0 || carry > 0) {
        int sum = carry; // Pehle carry ko sum mein daal lo

        // Agar num1 mein digits bache hain toh add karo
        if (i >= 0) {
            sum += num1[i] - '0'; // Char to Int conversion
            i--;
        }

        // Agar num2 mein digits bache hain toh add karo
        if (j >= 0) {
            sum += num2[j] - '0'; // Char to Int conversion
            j--;
        }

        // Naya carry nikallo
        carry = sum / 10;

        // Answer string mein naya digit push karo (Int to Char convert karke)
        ans.push_back((sum % 10) + '0');
    }

    // Kyunki humne right-to-left add kiya hai, answer ulta bana hai
    reverse(ans.begin(), ans.end());
}
```

```
        return ans;
    }

    int main() {
        string n1 = "456";
        string n2 = "77";
        cout << "Sum: " << addStrings(n1, n2) << endl; // Output: 533
        return 0;
    }
```

🕒 Complexity Analysis

- **Time Complexity:** $O(\max(N, M))$ — Jahan N aur M dono strings ki lengths hain. Loop utni hi baar chalega jitne bade dono mein se jo sabse bada number hoga uske digits honge. Yeh absolute best linear time hai.
- **Space Complexity:** $O(1)$ — Agar hum output string ko chhod dein (jo ki answer return karne ke liye zaroori hi hai), toh humne koi extra array ya data structure use nahi kiya hai, bas kuch variables liye hain.

Bhai, kya is approach mein jo `char` se `int` badalne ke liye hum - '0' karte hain, uska logic bilkul sahi se samajh aaya ya usme koi doubt hai?

Bhai, factorial nikalna toh programming ki duniya ka sabse purana aur popular sawal hai. Iska brute force tarika seedha mathematically jo hum paper par karte hain, usi ko code mein utarna hai—yaani ek loop chalana aur numbers ko aapas mein multiply karte jana.

Chalo iska poora logic, dry run, aur code bilkul detail mein samajhte hain.

🎯 Factorial Kya Hota Hai? (Intuition)

Kisi bhi positive number N ka factorial (jise hum $N!$ likhte hain) ka matlab hota hai 1 se lekar us number N tak ke saare integers ka aapas mein multiplication.

$$N! = N \times (N - 1) \times (N - 2) \times \dots \times 1$$

Example: Agar humein $5!$ (5 ka factorial) nikalna hai:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

💡 Brute Force Logic (Loop Method)

Brute force approach mein hum ek loop chalate hain aur ek accumulator variable (jaise `ans` ya `fact`) rakhte hain jisme shuruat mein 1 hota hai.

1. Ek variable banao: `long long fact = 1;` (Yahan `long long` isliye le rahe hain kyunki factorial bohot jaldi bada ho jata hai aur normal `int` se baahar nikal jata hai overflow hokar).
2. Ek loop chalao `i = 1` se lekar `N` tak.
3. Har iteration mein `fact = fact * i` karte jao.

4. Loop khatam hone par `fact` return kar do.

Dry Run (Example: $N = 4$)

Shuruat mein: `fact = 1`

- **i = 1:** `fact = fact * 1` $\rightarrow 1 * 1 = 1$
- **i = 2:** `fact = fact * 2` $\rightarrow 1 * 2 = 2$
- **i = 3:** `fact = fact * 3` $\rightarrow 2 * 3 = 6$
- **i = 4:** `fact = fact * 4` $\rightarrow 6 * 4 = 24$

Loop khatam! Final Answer = **24**

C++ Code (Iterative Brute Force)

```
#include <iostream>
#include <string>
using namespace std;

// Function to calculate factorial
long long getFactorial(int n) {
    // Edge Case: 0! aur 1! dono hamesha 1 hote hain
    if (n == 0 || n == 1) {
        return 1;
    }

    long long fact = 1; // Accumulator variable

    // Brute force loop from 2 to n (1 se multiply karne ka koi fayda nahi hota)
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    return fact;
}

int main() {
    int n = 5;
    if (n < 0) {
        cout << "Negative numbers ka factorial real numbers mein nahi hota bhai!"
        << endl;
    } else {
        cout << n << " ka Factorial hai: " << getFactorial(n) << endl;
    }
    return 0;
}
```

⚠️ Handle Karne Waale Edge Cases

1. $N = 0$ aur $N = 1$: Mathematically $0! = 1$ aur $1! = 1$ hota hai. Isliye code ke shuru mein hi check laga dena chahiye taaki loop chalane ki zaroorat hi na pade.
2. **Negative Numbers**: Negative numbers ka factorial simple mathematics mein define nahi hota (unke liye Gamma function use hota hai complex math mein). Isliye input check karna zaroori hai.
3. **Integer Overflow**: Factorial ki value bohot tezi se badhti hai. Ek normal 32-bit `int` max $12!$ tak hi jhel sakta hai. `long long` max $20!$ tak jhel sakta hai. Agar aapko isse bade number (jaise $100!$) ka factorial nikalna hai, toh aapko **String ya Vector array** use karke bada multiplication ka logic likhna padega (jise Big Integer approach kehte hain).

🕒 Time aur Space Complexity

- **Time Complexity**: $O(N)$ — Kyunki hum 1 se lekar N tak ka loop chala rahe hain, toh total N baar multiplication operations ho rahe hain.
- **Space Complexity**: $O(1)$ — Humne koi extra space, array, ya memory stack use nahi kiya hai. Sirf ek `fact` variable liya hai jo constant space consume karta hai.

Bhai, tumne bilkul sahi dukhti rag par haath rakha hai! Pichle answer mein jo maine **Integer Overflow** ki dikkat batayi thi, yeh uska absolute ultimate solution hai.

Jab $N = 100$ jaisa bada number hota hai, toh $100!$ mein **158 digits** hote hain! Itna bada number duniya ke kisi standard integer data type (`int`, `long long`, `__int128`) mein fit nahi ho sakta.

Isse nipatne ke liye hum **"Big Integer"** ka logic lagate hain, jahan hum pooray number ko ek data structure (jaise `vector<int>` ya `string`) mein **ek-ek digit karke ulta (reverse order mein)** store karte hain aur school waale multiplication ko simulate karte hain.

💡 Core Intuition: Pen-Paper Waala Multiplication

Jab humein 24 ko 5 se multiply karna hota hai, toh hum paper par kaise karte hain?

1. Hum pehle right side se shuru karte hain: $4 \times 5 = 20$.
2. 0 ko neeche likhte hain aur 2 ko **Carry** bhej dete hain.
3. Fir agla digit uthate hain: $2 \times 5 = 10$. Isme carry jodte hain: $10 + 2 = 12$.
4. 2 ko neeche likhte hain aur 1 ko fir carry bhejte hain. Kyunki aage koi digit nahi hai, toh 1 ko bhi neeche utar dete hain. Answer banta hai **120**.

Humein code mein bhi yahi karna hai! Hum ek `vector` lenge jo hamare factorial ke answer ke digits ko hold karega. Hum is vector mein digits ko **ulta** (`[4, 2]` yaani 24) isliye rakhenge, taaki jab naye digits (jaise carry) aayein, toh unhe peeche `push_back()` karna aasan ho (kyunki vector ke aage insert karna costly hota hai).

🔍 Dry Run (Kaise badhega array?)

Chalo dekhte hain ki $5!$ nikalte waqt array kaise kaam karega:

- **Shuruat mein**: `res = [1]` (Kyunki $1! = 1$)

- **Multiply by 2:** $1 \times 2 = 2 \rightarrow \text{res} = [2]$
- **Multiply by 3:** $2 \times 3 = 6 \rightarrow \text{res} = [6]$
- **Multiply by 4:** $6 \times 4 = 24 \rightarrow \text{res} = [4, 2]$ (Kyunki $24 \% 10 = 4$ aur carry 2 peeche chala gaya)
- **Multiply by 5 (24×5):**
 - Pehla element 4: $4 \times 5 + 0$ (carry) = 20 $\rightarrow$ Element bana $20 \% 10 = 0$, Carry gaya $20 / 10 = 2$.
res abhi hai: $[0, 2]$
 - Dusra element 2: $2 \times 5 + 2$ (carry) = 12 $\rightarrow$ Element bana $12 \% 10 = 2$, Carry gaya $12 / 10 = 1$.
res abhi hai: $[0, 2]$
 - Loop khatam, par carry 1 bacha hai $\rightarrow$ use `push_back()` kar do.
 - Final res array bana: $[0, 2, 1]$
 - **Answer Builder:** Jab is array ko ulta padhenge, toh yeh ban jayega "120".

C++ Code (Factorial of Large Number)

Yahan iska sabse clean aur production-grade C++ code hai jo 100! ya 500! bhi kuch milliseconds mein nikal dega:

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

// Yeh helper function ek bade number (jo vector mein hai) ko
// ek single normal integer 'x' se multiply karega
void multiply(int x, vector<int>& res) {
    int carry = 0;

    // Vector ke har ek digit ko 'x' se multiply karo
    for (int i = 0; i < res.size(); i++) {
        int prod = res[i] * x + carry;

        res[i] = prod % 10; // Sahi digit ko usi jagah bithaao
        carry = prod / 10; // Naya carry nikal lo
    }

    // Agar loop khatam hone ke baad bhi carry bacha hai,
    // toh use vector ke peeche ek-ek digit karke jodte jao
    while (carry) {
        res.push_back(carry % 10);
        carry = carry / 10;
    }
}
```

```
// Main function jo bade numbers ka factorial nikal kar string return karega
string largeFactorial(int n) {
    vector<int> res;
    res.push_back(1); // Base case: 1! = 1

    // Brute force multiplication loop 2 se lekar N tak
    for (int i = 2; i <= n; i++) {
        multiply(i, res);
    }

    // Kyunki digits ulte stored hain, string banane ke liye
    // hamein peeche se shuru karke characters jodne honge
    string factorial_str = "";
    for (int i = res.size() - 1; i >= 0; i--) {
        factorial_str += to_string(res[i]);
    }

    return factorial_str;
}

int main() {
    int n = 100; // Tum yahan 100, 200, ya 500 kuch bhi daal sakte ho!

    cout << n << "! ka answer hai: \n" << largeFactorial(n) << endl;

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$ — Hum ek loop chala rahe hain 2 se N tak ($O(N)$). Aur har step par hum `multiply` function call kar rahe hain, jo vector ke saare digits par chalta hai. $100!$ mein lagbhag 158 digits hote hain, toh digits ki length bhi lagbhag N ke proportional ghumti hai. Isliye total time $O(N \times \text{digits}) \approx O(N^2)$ ho jata hai. $N = 100$ ke liye yeh mushkil se 10,000 operations lega, jo palke jhapakte ही khatam ho jayenge!
- **Space Complexity:** $O(\text{digits})$ — Humne sirf factorial ke digits ko store karne ke liye vector banaya hai. Jaise $100!$ ke liye vector ki size 158 hogi, jo memory ke mamle mein bohot hi chhoti aur safe hai.

Bhai, yeh logic dekh kar interviewer samajh jata hai ki aapko memory limitations aur unke smart workarounds ki gehri samajh hai!

Bhai, yeh Prime Factorization waali approach computer science aur mathematics ka ek aisa masterstroke hai jo aapko normal programmers se alag khada karta hai.

Chalo bilkul shuruat se samajhte hain ki mathematicians ne iske baare mein **kya socha, kaise socha**, aur yeh concept aaya kahan se. Puri theory makkhan ki tarah clear karte hain.

The Spark: Dimag Mein Pehli Baat Kya Aayi?

Purane tarike mein dikkat yeh thi ki hum 2 se lekar N tak ke har number ko line se multiply kar rahe the. Kuch log bade the, kuch chote.

Mathematicians ne socha: "**Hum composite numbers (jaise 4, 6, 8, 9, 12) ko alag se multiply karne ki mehnat kyun kar rahe hain?**"

- 4 se multiply karne ka matlab hai: 2 se do baar multiply karna (2×2).
- 6 se multiply karne ka matlab hai: ek baar 2 aur ek baar 3 se multiply karna (2×3).
- 12 se multiply karne ka matlab hai: do baar 2 aur ek baar 3 se multiply karna ($2 \times 2 \times 3$).

Iska matlab, duniya ka koi bhi factorial ho, woh aakhiri mein banta toh sirf **Prime Numbers** (2, 3, 5, 7, 11...) ke multiplication se hi hai! Kisi bhi $N!$ ko hum aise likh sakte hain:

$$N! = 2^a \times 3^b \times 5^c \times 7^d \times \dots \times p^k$$

Yahan hamara naya target set hua: **Humein bas yeh pata lagana hai ki kaun sa Prime Number kitni baar (yaani uski power a, b, c kya hogi) is factorial mein aayega.**

Chote Example Se Soch ko Sahi Karte Hain (10!)

Maan lo humein 10! nikalna hai:

$$10! = 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$$

Agar hum har number ko uske prime factors mein tod dein:

- $2 = 2^1$
- $3 = 3^1$
- $4 = 2^2$
- $5 = 5^1$
- $6 = 2 \times 3$
- $7 = 7^1$
- $8 = 2^3$
- $9 = 3^2$
- $10 = 2 \times 5$

Ab agar hum saare 2 ki powers ko ek saath jodein: $1 + 2 + 1 + 3 + 1 = 8$ (Yaani 2^8) Saare 3 ki powers ko jodein: $1 + 1 + 2 = 4$ (Yaani 3^4) Saare 5 ki powers ko jodein: $1 + 1 = 2$ (Yaani 5^2) Saare 7 ki powers ko jodein: 1 (Yaani 7^1)

Toh 10! ka asli roop kya bana?

$$10! = 2^8 \times 3^4 \times 5^2 \times 7^1$$

Legendre's Formula Kaise Aaya? (The Core Discovery)

Ab problem yeh thi ki agar $N = 10000$ ho, toh hum aise ek-ek karke prime numbers thodi na ginte baithenge! Yahan entry hoti hai **Legendre's Formula** ki.

Unhone ek kamaal ka pattern notice kiya. Maan lo humein pata karna hai ki 10! mein 2 kitni baar aata hai:

- Step 1:** 1 se 10 tak ke beech mein har dusra number 2 se divide hota hai. Toh kitne numbers hain? $\lfloor 10/2 \rfloor = 5$ numbers (jo hain: 2, 4, 6, 8, 10). In sabke paas kam se kam ek 2 toh hai hi. (Total Count = 5)
- Step 2:** Lekin kuch numbers aise hain jo 4 (2^2) se divide hote hain, yaani unke paas ek extra 2 bhi hai! Aise kitne numbers hain? $\lfloor 10/4 \rfloor = 2$ numbers (jo hain: 4 aur 8). (Total Count = 5 + 2 = 7)
- Step 3:** Kuch numbers aise hain jo 8 (2^3) se divide hote hain, yaani unke paas ek aur extra 2 hai! Aise kitne numbers hain? $\lfloor 10/8 \rfloor = 1$ number (jo hai: 8). (Total Count = 7 + 1 = 8)
- Step 4:** Agla step hoga $\lfloor 10/16 \rfloor = 0$, toh hum ruk gaye.

Dhyan se dekho, bina kisi loop ke, sirf division karke humne nikal liya ki 2 ki power 8 hogi! Yahi hai Legendre's Formula:

$$E_p(N!) = \sum_{i=1}^{\infty} \left\lfloor \frac{N}{p^i} \right\rfloor = \left\lfloor \frac{N}{p} \right\rfloor + \left\lfloor \frac{N}{p^2} \right\rfloor + \left\lfloor \frac{N}{p^3} \right\rfloor + \dots$$

Yeh Approach Sabse Fast Kyun Bani? (The Ultimate Strategy)

Jab humein saare prime numbers ki exact powers pata chal gayi, toh computer ke liye kaam makkhan kyun ho gaya?

- Sieve of Eratosthenes:** 1 se N tak ke saare prime numbers nikalna computer ke liye milliseconds ka kaam hai.
- Binary Exponentiation:** Agar humein 2^{80} nikalna hai, toh hum 2 ko 80 baar multiply nahi karte. Hum computer mein $O(\log \text{power})$ waala binary exponentiation lagate hain, jo chutkiyon mein bade-bade powers calculate kar deta hai.
- Fewer Big Multiplications:** Purane tarike mein hum N baar bade-bade strings ko multiply kar rahe the. Is tarike mein hamare paas sirf kuch gine-chune prime numbers bachte hain (Prime Number Theorem ke mutabik N tak sirf $\approx \frac{N}{\log N}$ primes hote hain). Humein bas unke beech mein hi final multiplication karna padta hai.

Summary

- Kya socha?** Numbers ko sequentially multiply karne ke bajaye unke sabse chote tukde (Primes) mein tod kar socha.
- Kaise aaya?** Legendre ne ginti karne ka ek shortcut formula dhoondha ($\lfloor N/p \rfloor$), jisse bina loop chalaye kisi bhi prime ki total power mil jati hai.
- Fayda kya hua?** Faltu ke hazaron multiplications khatam ho gaye, aur computer sirf kaam ki cheezon (Primes) par focus karne laga.

Bhai, kya yeh mathematical intuition aur Legendre ka ginti karne ka tarika aapko पूरी तरह samajh aaya, ya iske kisi step par thoda aur gehraai mein jaana hai?

Bhai, tumhara optimization ka keeda sach mein kamaal ka hai! Haan, isko **aur zyada optimize kiya ja sakta hai**.

Pichle code mein dikkat kya thi? Hum ek bohot bade number ko ek chhote number (i) se baar-baar multiply kar rahe the. Isko kehte hain **Linear Sequential Multiplication**. Jab number bada hota jata hai, toh har naye multiplication mein poore array par loop chalana padta hai, jisse unbalance create hota hai.

Isse nipatne ke liye hamare paas **do advanced tarike** hain:

1. **Divide and Conquer (Binary Splitting)** — Jo standard programming mein use hota hai.
2. **Prime Factorization (Legendre's Formula)** — Jo competitive programming ka ultimate math hack hai.

Chalo pehle **Divide and Conquer** waale tarike ko detail mein samajhte hain, kyunki yeh coding interviews ke liye sabse practical aur dhasu approach hai.

Intuition: Divide and Conquer (Binary Splitting)

Maan lo humein 1 se 8 tak ka factorial nikalna hai.

- **Purana Tarika:** ((((((1*2)*3)*4)*5)*6)*7)*8 → Ek bada number banta ja raha hai aur chote number se multiply ho raha hai.
- **Naya Tarika:** Hum range ko do hisson mein tod denge (tree ki tarah).

Hum (1*2*3*4) ka result alag nikalenge aur (5*6*7*8) ka alag nikalenge. Phir un dono bade numbers ko aapas mein multiply karenge.

Iska Fayda Kya Hai? Jab do lagbhag barabar size ke bade numbers aapas mein multiply hote hain, toh modern computer systems aur advanced multiplication algorithms (jaise Karatsuba ya FFT) unhe bohot tezi se solve kar paate hain. Isse loop ka overhead bohot kam ho jata hai.

Optimized C++ Code (Divide & Conquer Approach)

Isme humein do bade numbers (strings) ko aapas mein multiply karne ka logic chahiye hoga (jo ki khud LeetCode ka ek famous question hai: *Multiply Strings*).

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

// Helper Function: Do bade numbers (strings) ko multiply karne ke liye
string multiplyStrings(string num1, string num2) {
    if (num1 == "0" || num2 == "0") return "0";

    int n = num1.size(), m = num2.size();
    vector<int> result(n + m, 0);

    // School method multiplication backward direction mein
    for (int i = n - 1; i >= 0; i--) {
        for (int j = m - 1; j >= 0; j--) {
            int mul = (num1[i] - '0') * (num2[j] - '0');
            int sum = mul + result[i + j + 1];

            result[i + j + 1] = sum % 10;
            result[i + j] += sum / 10;
        }
    }
}
```

```

    }

    // Vector ko wapas string mein convert karna
    string ans = "";
    int i = 0;
    while (i < result.size() && result[i] == 0) i++; // Leading zeros hatao
    while (i < result.size()) ans += to_string(result[i++]);

    return ans;
}

// Main Divide and Conquer Function
string factorialHelper(int start, int end) {
    if (start > end) return "1";
    if (start == end) return to_string(start);

    // Range ko beech se aadha todo
    int mid = start + (end - start) / 2;

    // Left half aur Right half ka answer recursively nikalo
    string leftResult = factorialHelper(start, mid);
    string rightResult = factorialHelper(mid + 1, end);

    // Dono bade numbers ko aapas mein multiply kar do
    return multiplyStrings(leftResult, rightResult);
}

string largeFactorialOptimized(int n) {
    if (n == 0 || n == 1) return "1";
    return factorialHelper(2, n);
}

int main() {
    int n = 100;
    cout << n << "! ka Highly Optimized Answer:\n" << largeFactorialOptimized(n)
    << endl;
    return 0;
}

```

Time Complexity Kaise Sudhri?

- **Pehle Waala Brute Force:** $O(N^2)$ chal raha tha kyunki har step par ek bada number ek chote digit se multiply ho raha hai.
- **Yeh Waala Divide & Conquer:** Agar hum standard multiplication use kar rahe hain toh yeh overhead ko kam karta hai. Lekin agar iske saath **Karatsuba Algorithm** joda jaye, toh iski time complexity ghatkar lagbhag $O(N \log^2 N)$ ya $O(N \log^3 N)$ ke zone mein aa jaati hai, jo $O(N^2)$ se lakho guna tez hai bade inputs ($N = 10000+$) ke liye.

Bonus: The Ultimate Math Boss (Prime Factorization)

Agar tumse koi pooch le ki "Mujhe isse bhi tez chahiye, bina multiply kiyel!", toh unhe bolna: **Legendre's Formula**.

Hum jaante hain ki kisi bhi number ke factorial ($N!$) ko uske prime factors ke roop mein toda ja sakta hai:

$$N! = 2^a \times 3^b \times 5^c \times 7^d \dots$$

1. Hum N se chhote saare prime numbers dhoondte hain (Sieve of Eratosthenes se).
2. Har prime number ki power ($a, b, c \dots$) nikalte hain Legendre's formula se: $E_p(N) = \lfloor N/p \rfloor + \lfloor N/p^2 \rfloor + \dots$
3. Phir un prime numbers ko unki powers ke saath **Binary Exponentiation** ($O(\log \text{power})$) se calculate karke aapas mein multiply kar dete hain.

Yeh approach mathematically sabse zyada optimized hai kyunki yeh beech ke kisi bhi faltu composite number (jaise 4, 6, 8, 9) ko multiply karne ka samay bacha leti hai!